# Weight Clipping versus Gradient Penalty in Wasserstein GAN Training

**Antonio Peña Peña** **Geer Wang Liu** **Zhenhua Ye Xiang**

## Abstract

Las redes generativas adversarias (GAN) pueden generar imágenes de gran calidad, pero su entrenamiento puede ser inestable. Las Wasserstein GAN (WGAN) contribuyen a mejorar esta estabilidad, aunque Gulrajani et al. [1] observan que el recorte de pesos utilizado para imponer la restricción de Lipschitz al crítico puede provocar dificultades de optimización, muestras de baja calidad o falta de convergencia. Como alternativa, proponen WGAN-GP, que incorpora una penalización para favorecer que la norma del gradiente del crítico respecto a su entrada se aproxime a 1 en puntos interpolados entre datos reales y generados. En este trabajo replicamos parcialmente esa comparación: entrenamos ambos modelos en PyTorch sobre CIFAR-10, con la misma arquitectura del generador y críticos de estructura similar, aunque con diferencias en la normalización del crítico y en el optimizador. Comparamos la estabilidad del entrenamiento y la calidad de las imágenes generadas mediante el análisis de curvas de pérdida de entrenamiento, normas de gradientes, métricas de calidad y diversidad e inspección visual. Los resultados muestran que WGAN requiere menos tiempo de entrenamiento, mientras que WGAN-GP genera imágenes más reconocibles en etapas más tempranas y obtiene mejores resultados cuantitativas al finalizar.

## 1. Introducción

Las redes generativas adversarias (GAN) son una familia de modelos generativos capaces de producir imágenes a partir de ruido [2]. Están formadas por dos redes: un generador, que produce datos sintéticos a partir de una fuente de ruido, y un discriminador, que distingue entre la salida del generador y los datos reales.

Aunque pueden generar imágenes muy convincentes, su entrenamiento adversarial es a menudo inestable y puede sufrir gradientes que se desvanecen, oscilaciones y colapso de modos. Para abordar estas dificultades, Arjovsky et al. [3] propusieron las Wasserstein GAN (WGAN), que sustituyen el objetivo clásico por uno basado en la distancia de Wasserstein-1. Su formulación teórica introduce un crítico de salida real sujeto a una restricción 1-Lipschitz. En la práctica, el método original recorta los pesos del crítico al intervalo $[-c, c]$ para controlar su constante de Lipschitz, aunque este procedimiento no garantiza que dicha constante sea exactamente 1.

Gulrajani et al. [1] observaron que este recorte puede dar lugar a un comportamiento indeseado del crítico y propusieron WGAN con penalización de gradiente (WGAN-GP). Este método añade a la pérdida del crítico un término que favorece que la norma de su gradiente respecto a la entrada se aproxime a 1 en puntos interpolados entre datos reales y generados. Según sus experimentos, esta alternativa permite un entrenamiento más estable sin necesidad de recortar los pesos.

En este trabajo realizamos una réplica parcial de esa comparación. Debido a las limitaciones de tiempo y recursos computacionales, no reproducimos todos los experimentos del artículo, sino que centramos el estudio en comparar WGAN y WGAN-GP sobre CIFAR-10. Ambos modelos utilizan la misma arquitectura del generador y críticos de estructura similar, aunque difieren en la norma-

lización del crítico, el optimizador y la tasa de aprendizaje. Por ello, las diferencias observadas no pueden atribuirse únicamente a la penalización de gradiente.

## 2. Objetivos

El objetivo es comparar WGAN con recorte de pesos y WGAN-GP sobre CIFAR-10 mediante una implementación en PyTorch. Se busca evaluar la estabilidad del entrenamiento, el comportamiento de los gradientes del crítico respecto a su entrada y la calidad de las imágenes generadas, y contrastar las observaciones con el artículo original.

La hipótesis es que la configuración de WGAN-GP mostrará un entrenamiento más estable y mejores muestras, con normas del gradiente más próximas a 1 en los puntos interpolados donde se aplica la penalización, en línea con Gulrajani et al. [1]. La comparación se interpretará considerando las diferencias de configuración descritas en la Sección 4 y el alcance parcial de la réplica.

## 3. Trabajo relacionado y estado del arte

**GAN y su inestabilidad.** Las GAN plantean un juego minimax entre un generador $G$ y un discriminador $D$ [2]:

$$\min_G \max_D \; \mathbb{E}_{x\sim P_r}[\log D(x)] + \mathbb{E}_{\tilde{x}\sim P_g}[\log(1 - D(\tilde{x}))], \tag{1}$$

donde $P_r$ es la distribución de los datos reales y $P_g$ es la distribución del modelo, definida implícitamente por $\tilde{x} = G(z)$ con $z \sim p(z)$, una distribución sencilla como una uniforme o una gaussiana. Si el discriminador se entrena hasta la optimalidad, minimizar la función de valor respecto al generador equivale a minimizar la divergencia de Jensen–Shannon entre $P_r$ y $P_g$. En la práctica, el objetivo minimax del generador puede proporcionar gradientes débiles cuando el discriminador distingue con facilidad las muestras reales de las generadas [1]. Goodfellow et al. [2] proponen entrenar al generador para maximizar $\mathbb{E}_{\tilde{x}\sim P_g}[\log D(\tilde{x})]$, pero incluso esta variante puede comportarse mal ante un buen discriminador [4].

Radford et al. [7] propusieron DCGAN, una arquitectura convolucional para GAN que Gulrajani et al. [1] toman como punto de partida en varios de sus experimentos. Salimans et al. [6] presentan varias técnicas para mejorar el entrenamiento, como *feature matching* o *minibatch discrimination*, y proponen el Inception Score para evaluar la calidad de las muestras. Mao et al. [5] proponen LSGAN, que sustituye la entropía cruzada con sigmoide del discriminador por una pérdida de mínimos cuadrados y con la que, según sus autores, se obtienen imágenes de mayor calidad y un aprendizaje más estable. Además, siguiendo a DCGAN, usan ReLU en las capas ocultas del generador y LeakyReLU en las del discriminador, igual que nuestros modelos.

**Wasserstein GAN.** Arjovsky et al. [3] argumentan que las divergencias que suelen minimizar las GAN pueden no ser continuas respecto a los parámetros del generador, y proponen en su lugar la distancia *Earth-Mover*, también llamada Wasserstein-1, $W(q, p)$. Informalmente, esta distancia es el coste mínimo de transportar masa para transformar la distribución $q$ en la distribución $p$, con un coste igual a la masa transportada multiplicada por la distancia de transporte. Bajo condiciones de regularidad, la distancia $W(P_r, P_{g_\theta})$ es continua y diferenciable casi en todas partes respecto a los parámetros $\theta$ del generador [3, 1]. Su función de valor se obtiene mediante la dualidad de Kantorovich–Rubinstein [9]:

$$\min_G \max_{D\in\mathcal{D}} \; \mathbb{E}_{x\sim P_r}[D(x)] - \mathbb{E}_{\tilde{x}\sim P_g}[D(\tilde{x})], \tag{2}$$

donde $\mathcal{D}$ es el conjunto de funciones 1-Lipschitz. Con un crítico óptimo, minimizar esa función respecto a los parámetros del generador equivale a minimizar $W(P_r, P_g)$. Se denomina crítico y no discriminador porque asigna puntuaciones reales en lugar de entrenarse para clasificar las muestras como reales o generadas.

En la práctica, el crítico maximiza esa diferencia, lo que equivale a minimizar su signo contrario, $\mathbb{E}_{\tilde{x}\sim P_g}[D(\tilde{x})] - \mathbb{E}_{x\sim P_r}[D(x)]$, que es la pérdida que usamos en la Sección 4. Manteniendo fijo el crítico durante la actualización del generador, este minimiza $-\mathbb{E}_{z\sim p(z)}[D(G(z))]$, que es la parte del objetivo que depende de él.

Para controlar la constante de Lipschitz, el método original recorta los pesos del crítico a $[-c, c]$. Esto lo limita a un subconjunto de las funciones $k$-Lipschitz, con un $k$ que depende de $c$ y de la arquitectura, sin garantizar que sea exactamente 1 [1]. El objetivo de WGAN proporciona gradientes más adecuados para optimizar el generador y, según las observaciones de los autores, su valor se correlaciona con la calidad de las muestras [3, 1].

***Weight clipping* frente a *gradient penalty*.** Gulrajani et al. [1] muestran que el recorte de pesos puede limitar la capacidad del crítico, favoreciendo funciones demasiado simples, y provocar gradientes que se desvanecen o explotan. Como alternativa, proponen WGAN-GP, que añade a la pérdida del crítico una penalización para favorecer que la norma de su gradiente respecto a la entrada se aproxime a 1 en puntos interpolados entre muestras reales y generadas. La penalización es bilateral: penaliza normas superiores e inferiores a 1, aunque no garantiza que el crítico sea 1-Lipschitz en todo el espacio.

Los autores utilizan $\lambda = 10$ y omiten la *batch normalization* en el crítico, porque introduce dependencias entre las muestras del lote y la penalización se calcula para cada entrada de forma independiente. Como alternativa, recomiendan *layer normalization*, que no introduce estas dependencias [1, 10].

En sus experimentos sobre CIFAR-10, WGAN-GP converge más rápido y alcanza un IS mayor que WGAN con recorte, incluso utilizando el mismo optimizador RMSProp y la misma tasa de aprendizaje. Una configuración con Adam y una tasa de aprendizaje mayor mejora adicionalmente el rendimiento [1]. Estos resultados motivan la comparación parcial que realizamos en este trabajo.

**Métricas.** Para evaluar la calidad de las muestras se consideran cinco métricas complementarias.

- Fréchet Inception Distance (FID)[11]: mide la distancia entre las distribuciones de las imágenes reales y generadas en el espacio de características de una red Inception; valores más bajos indican mayor similitud.
- Kernel Inception Distance (KID)[6]: compara ambas distribuciones mediante una función kernel aplicada a sus características, donde valores más bajos reflejan una mayor similitud.
- Inception Score (IS): evalúa la calidad y diversidad de las imágenes a partir de las predicciones de una red Inception; valores más altos suelen indicar mejores resultados.
- Recall: mide hasta qué punto las muestradas generadas cubren la variedad de distribución de imágenes reales; valores más altos indican una mayor cobertura de la distribución real.
- Coverage: estima la proporción de imágenes reales que quedan representadas por las muestras generadas; valores más altos indican una mayor cobertura de la distribución real.

## 4. Metodología

Se implementan en PyTorch WGAN con recorte de pesos y WGAN-GP con penalización de gradiente sobre CIFAR-10. Ambos métodos utilizan la misma arquitectura del generador y críticos de estructura similar, aunque difieren en la normalización del crítico, el optimizador y la tasa de aprendizaje. Por tanto, la comparación no permite atribuir las diferencias exclusivamente a la penalización de gradiente.

### 4.1. Preparación de los datos

Se utiliza el conjunto de entrenamiento de CIFAR-10 [8], con 50 000 imágenes RGB de $32 \times 32$ píxeles, sin emplear sus etiquetas durante el entrenamiento. Las imágenes se convierten a tensores y sus valores se transforman de $[0, 1]$ a $[-1, 1]$ mediante $x' = (x - 0{,}5)/0{,}5$, de acuerdo con el rango de salida de la función $\tanh$ del generador.

Tabla 1: Arquitecturas implementadas. ConvT indica convolución transpuesta y BN, batch normalization. [BN] aparece únicamente en el crítico de WGAN. Las dimensiones excluyen el tamaño del lote.

| Red | Capa u operación | Salida |
|---|---|---|
| Generador | Lineal, BN, ReLU | 8192 |
| | Reorganización | $512 \times 4 \times 4$ |
| | ConvT, BN, ReLU | $256 \times 8 \times 8$ |
| | ConvT, BN, ReLU | $128 \times 16 \times 16$ |
| | ConvT, $\tanh$ | $3 \times 32 \times 32$ |
| Crítico | Conv, LeakyReLU | $128 \times 16 \times 16$ |
| | Conv, [BN], LeakyReLU | $256 \times 8 \times 8$ |
| | Conv, [BN], LeakyReLU | $512 \times 4 \times 4$ |
| | Aplanado y lineal | 1 |

Tabla 2: Configuración de ambos métodos. El optimizador y la tasa de aprendizaje se aplican a ambas redes de cada método.

| Configuración | WGAN | WGAN-GP |
|---|---|---|
| Tamaño de lote | 64 | 64 |
| Dimensión del ruido | 128 | 128 |
| Anchura base (`dim`) | 128 | 128 |
| Actualizaciones de $D$ por cada una de $G$ | 5 | 5 |
| Presupuesto de actualizaciones de $G$ | 200 000 | 200 000 |
| Optimizador | RMSprop | Adam |
| Tasa de aprendizaje | $5 \cdot 10^{-5}$ | $10^{-4}$ |
| $(\beta_1, \beta_2)$ de Adam | No aplica | (0,5, 0,9) |
| Umbral de recorte $c$ | 0.01 | No aplica |
| Penalización $\lambda$ | No aplica | 10 |
| Normalización del crítico | BN | Ninguna |

Se configuran lotes de 64 imágenes con `shuffle=True` y `drop_last=True`, por lo que se descarta el último lote incompleto. El código utiliza `cycle(train_loader)`, que repite los lotes de la primera pasada sin renovar su barajado en cada vuelta.

### 4.2. Arquitecturas

Se emplean las clases `Generator`, `Critic` y `CriticGP`. El generador recibe ruido $z \sim \mathcal{N}(0, I_{128})$ y produce imágenes mediante una arquitectura convolucional de estilo DCGAN [7]. El crítico transforma cada imagen en una puntuación escalar sin sigmoide.

La Tabla 1 resume las capas. Todas las convoluciones utilizan núcleo $5 \times 5$, *stride* 2 y *padding* 2; las transpuestas emplean además `output_padding=1`. LeakyReLU utiliza una pendiente negativa de 0.2.

En WGAN, el crítico incorpora *batch normalization* en sus dos últimos bloques convolucionales. En WGAN-GP no se utiliza normalización en el crítico, para evitar dependencias entre las muestras al calcular la penalización de gradiente [1].

### 4.3. Configuración del entrenamiento

Se crean redes independientes para cada método y se configura un presupuesto de 200 000 actualizaciones del generador, con cinco actualizaciones del crítico por cada una.

La configuración de Adam coincide con la del Apéndice E de Gulrajani et al. [1]; se utiliza $\beta_1 = 0{,}5$, frente al valor por defecto $\beta_1 = 0$ de su Algoritmo 1.

Cada iteración del generador incluye cinco actualizaciones del crítico, por lo que en total se realiza 200 000 actualizaciones del generador y 1000 000 del crítico por modelo. Los checkpoints permiten

recuperar los estados de las redes y los optimizadores, así como consultar los historiales de entrenamiento sin repetir las ejecuciones.

### 4.4. Funciones de pérdida

Se denotan por $\theta$ los parámetros del generador y por $w$ los del crítico. Para un lote de $m = 64$ imágenes reales $x_i$ y generadas $\tilde{x}_i = G_\theta(z_i)$, el crítico de WGAN minimiza

$$L_D = \frac{1}{m}\sum_{i=1}^{m} D_w(\tilde{x}_i) - \frac{1}{m}\sum_{i=1}^{m} D_w(x_i). \tag{3}$$

Después de cada actualización del crítico se recortan todos sus parámetros entrenables:

$$w \leftarrow \mathrm{clip}(w, -c, c), \qquad c = 0{,}01. \tag{4}$$

En el código, este paso se implementa mediante `p.data.clamp_(-0.01, 0.01)`.

En WGAN-GP se elimina el recorte y se construyen interpolaciones entre imágenes reales y generadas:

$$\hat{x}_i = x_i + \epsilon_i(\tilde{x}_i - x_i), \qquad \epsilon_i \sim U[0,1]. \tag{5}$$

Se utiliza un coeficiente por imagen, compartido por todos sus píxeles y canales. La pérdida del crítico pasa a ser

$$L_D^{GP} = L_D + \lambda\frac{1}{m}\sum_{i=1}^{m}\left(\|\nabla_{\hat{x}_i} D_w(\hat{x}_i)\|_2 - 1\right)^2, \qquad \lambda = 10. \tag{6}$$

Las funciones `interpolate_samples`, `gradient_penalty` y `wgan_gp_critic_loss` implementan estos pasos. El gradiente respecto a la entrada se obtiene mediante `torch.autograd.grad`, con `create_graph=True` para poder derivar la penalización respecto a los parámetros del crítico. La norma $L_2$ se calcula por imagen sobre sus $3 \cdot 32 \cdot 32 = 3072$ componentes.

En ambos métodos, el generador minimiza

$$L_G = -\frac{1}{m}\sum_{i=1}^{m} D_w(G_\theta(z_i)). \tag{7}$$

Durante esta actualización, el gradiente se propaga a través del crítico, pero solo se aplica el paso del optimizador del generador.

### 4.5. Procedimiento de entrenamiento

El Algoritmo 1 del Apéndice A.1 resume la secuencia común a ambos métodos. En cada actualización se reinician los gradientes del optimizador correspondiente mediante `zero_grad()`, se calculan los gradientes con `backward()` y se actualizan los parámetros con `step()`.

Durante las actualizaciones del crítico se impide la propagación de gradientes hacia el generador. En WGAN se utiliza `detach()` sobre las imágenes generadas y, en WGAN-GP, estas se producen dentro de un bloque `torch.no_grad()`.

Se registra una pérdida por cada actualización de cada red. En WGAN-GP se conservan además la componente Wasserstein y la penalización por separado.

En WGAN, se guarda el estado de entrenamiento cada 2 500 iteraciones, mientras que en WGAN-GP, debido a su mayor tiempo de entrenamiento, se almacena cada 1000 iteraciones. De estado manera, se puede recuperar el estado si se produce una interrupción inesperada. Para el análisis comparativo se conservan cuatro puntos clave: 1 000, 10 000, 100 000 y 200 000 iteraciones.

### 4.6. Evaluación

En primer lugar, se realiza una comparación visual de imágenes generadas en cada checkpoint. Para que la comparación sea consistente, ambos generadores reciben el mismo lote fijo de ruido y se muestran 64 imágenes por modelo, organizadas en una cuadrícula de 8x8.

Después, se estudia la estabilidad del entrenamiento con las curvas de pérdida y la distribución de las normas del grandiente del crítico. Para el análisis adicional de las normas del gradiente, se calcula por separado el gradiente de la puntuación del crítico respecto a la entrada de cada imagen real y se obtiene su norma; las medias, medianas y desviaciones estándar resumen la distribución en cada checkpoint.

Para evaluar cuantitativamente los modelos, se comparan las imágenes generadas en cada checkpoint con imágenes reales del conjunto de test de CIFAR-10. Se calculan FID, KID e IS mediante la librería `torch_fidelity`, y Recall y Coverage mediante la librería `prdc`.

## 5. Resultados y discusión

### 5.1. Evolución visual de las imágenes generadas

En la figura 4 del Apéndice A.2 se muestra la evolución de las imágenes generadas por los cuatro checkpoints por ambos modelos. En las primeras 1000 iteraciones, ambos modelos generan imágenes difusas y poco renocibles. A las 10 000 iteraciones, WGAN-GP presenta estructuras más definidas y reconocibles que WGAN, lo que sugiere una evolución inicial más rápida. A las 100 000 iteraciones, ambos modelos generan imágenes en las que comienzan a identificarse animales, vehículos y paisajes, aunque todavía presentan deformaciones y falta de nitidez.

En el checkpoint final, correspondiente a 200 000 iteraciones, las diferencias visuales entre ambos modelos son menores. La comparación sugiere que WGAN-GP aprende más rápidamente durante las primeras etapas, aunque no presenta una superioridad visual contundente al finalizar el entrenamiento.

### 5.2. Curvas de pérdidas

En WGAN, la pérdida del generador presenta una tendencia general descendente y termina estabilizándose en valores próximos a 0.1. Por su parte, la componente Wasserstein del crítico aumenta progresivamente desde valores negativos hasta aproximarse a -0.2. Este comportamiento indica que disminuye la diferencia entre las puntuaciones asignadas a las imágenes reales y generadas.

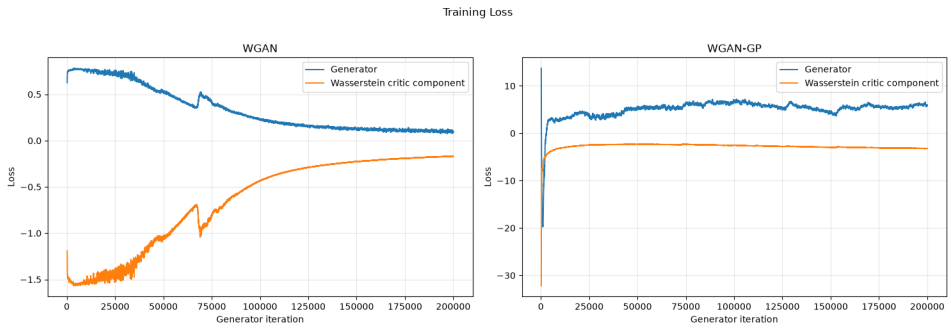



Figura 1: Curvas de pérdidas de WGAN y WGAN-GP

En WGAN-GP, la pérdida del generador presenta inicialmente valores elevados y posteriormente se mantiene en un intervalo más estable. La componente Wasserstein del crítico comienza con valores muy negativos y aumenta progresivamente hasta aproximarse a -3.

No obstante, aunque las curvas permiten estudiar la evolución interna de cada entrenamiento, no demuestran por sí solas que uno de los modelos sea más estable o genere imágenes de mayor calidad. Para ello es necesario considerar también las métricas y las distribuciones de los gradientes.

### 5.3. Penalización del gradiente en WGAN-GP

Según la figura 2, la penalización del gradiente comienza con valores relativamente elevados, aproximadamente 2.4, y disminuye rápidamente durante las primeras actualizaciones. Posteriormente permanece en valores pequeños, aproximadamente entre 0.02 y 0.05, con algunas fluctuaciones.

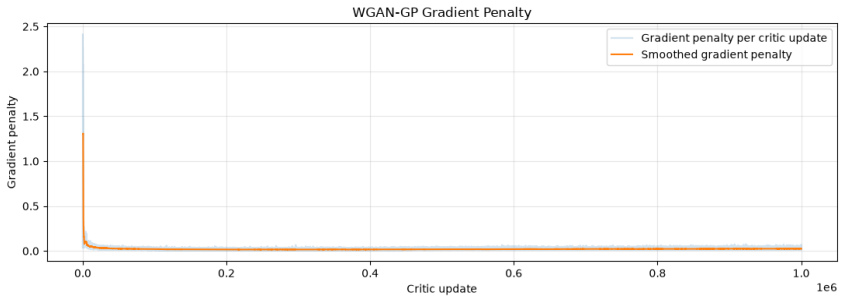



Figura 2: Curva de penalización del gradiente en WGAN-GP

La reducción de la penalización indica que las normas de los gradientes se aproximan a 1 en los puntos interpolados regularizados. Sin embargo, una penalización pequeña no garantiza la condición 1-Lipschitz en todo el espacio.

### 5.4. Distribución de las normas del gradiente del crítico

| Modelo | Iteraciones | Media | Mediana | Desviación estándar |
|---|---|---|---|---|
| WGAN | 1 000 | 0.0717 | 0.0718 | 0.0067 |
| WGAN-GP | 1 000 | 1.3641 | 1.3328 | 0.1950 |
| WGAN | 10 000 | 0.2768 | 0.2792 | 0.0205 |
| WGAN-GP | 10 000 | 1.1774 | 1.1728 | 0.0852 |
| WGAN | 100 000 | 1.0333 | 0.9641 | 0.3328 |
| WGAN-GP | 100 000 | 1.1226 | 1.1149 | 0.0738 |
| WGAN | 200 000 | 0,6182 | 0,5815 | 0,1467 |
| WGAN-GP | 200 000 | 1,1495 | 1,1456 | 0,0744 |

Tabla 3: Distribución de las normas del gradiente del crítico

En WGAN, la media de las normas del gradiente aumenta de 0,0717 a 1,0333 entre las 1 000 y las 100 000 iteraciones, pero desciende a 0,6182 en el checkpoint final. La desviación estándar alcanza 0,3328 a las 100 000 iteraciones. En WGAN-GP, las medias permanecen más próximas a 1 (1,3641, 1,1774, 1,1226 y 1,1495), mientras que la desviación estándar se reduce desde 0,1950 hasta valores próximos a 0,07–0,09. En conjunto, WGAN-GP presenta una distribución más concentrada en las imágenes reales evaluadas, en línea con su mecanismo de regularización.

### 5.5. Evaluación cuantitativa

En las siguientes graficas se muestran la evolución de las cinco métricas evaluadas:

**FID y KID** Ambas métricas disminuyen progresivamente en los dos modelos, lo que indica que las distribuciones de características de las imágenes generadas se aproximan a las de las imágenes reales. WGAN-GP presenta mejores valores en todos los checkpoints, aunque al finalizar las diferencias son pequeñas: el FID es 40.20 frente a 42.74 y el KID es 0.0265 frente a 0.0268.

**IS** Ambos modelos aumentan su puntuación durante el entrenamiento. WGAN-GP obtiene valores superiores en los cuatro checkpoints, aunque su puntuación disminuye ligeramente entre las 100 000 y las 200 000 iteraciones, pasando de 6.34 a 6.27.

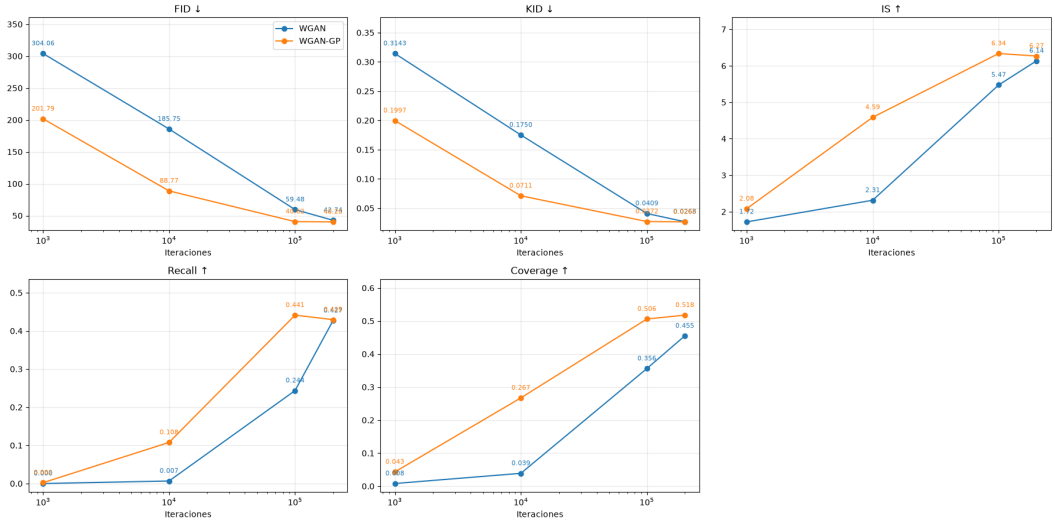



Figura 3: Gráfica de evolución de las métricas

**Recall** Esta métrica mejora considerablemente durante el entrenamiento. WGAN-GP presenta una ventaja importante en las etapas intermedias, alcanzando 0.4413 a las 100 000 iteraciones frente a 0.2436 de WGAN. Sin embargo, en el checkpoint final los valores son muy similares.

**Coverage** Esta métrica también aumenta durante el entrenamiento. En el checkpoint final, WGAN-GP obtiene 0.5176 frente a 0.4549 de WGAN. Esta diferencia sugiere que WGAN-GP cubre una mayor proporción de la distribución real en el espacio de características.

En conjunto, WGAN-GP obtiene mejores resultados en las cinco métricas en el checkpoint final. No obstante, las diferencias en FID, KID, IS y Recall son pequeñas, mientras que Coverage presenta una ventaja más apreciable.

## 6. Conclusiones y trabajo futuro

Los resultados muestran que WGAN-GP presenta una distribución más concentrada de las normas del gradiente evaluadas sobre imágenes reales y genera muestras más reconocibles en etapas tempranas. Al finalizar el entrenamiento, también obtiene mejores valores en FID, KID, IS, Recall y Coverage, aunque varias diferencias son reducidas y la ventaja más apreciable aparece en Coverage.

Por tanto, en las configuraciones estudiadas, WGAN-GP presenta un comportamiento general más favorable que WGAN con recorte de pesos. Sin embargo, las diferencias de arquitectura y optimización impiden atribuir estas mejoras exclusivamente al uso de gradient penalty. Además, al no disponer de ejecuciones con varias semillas, no puede estimarse la variabilidad entre entrenamientos.

Como trabajo futuro, sería interesante realizar una comparación más controlada utilizando configuraciones equivalentes, modificando únicamente el mecanismo de regularización, y repetir los experimentos con distintas semillas para obtener estimaciones más robustas.

## Referencias

[1] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville. Improved training of Wasserstein GANs. In *Advances in Neural Information Processing Systems 30*, 2017.

[2] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, pages 2672–2680, 2014.

[3] M. Arjovsky, S. Chintala, and L. Bottou. Wasserstein GAN. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.

[4] M. Arjovsky and L. Bottou. Towards principled methods for training generative adversarial networks. 2017.

[5] X. Mao, Q. Li, H. Xie, R. Y. K. Lau, Z. Wang, and S. P. Smolley. Least squares generative adversarial networks. *arXiv preprint arXiv:1611.04076*, 2016.

[6] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen. Improved techniques for training GANs. In *Advances in Neural Information Processing Systems*, pages 2226–2234, 2016.

[7] A. Radford, L. Metz, and S. Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434*, 2015.

[8] A. Krizhevsky. Learning multiple layers of features from tiny images. 2009.

[9] C. Villani. *Optimal Transport: Old and New*, volume 338. Springer Science & Business Media, 2008.

[10] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[11] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter. GANs trained by a two time-scale update rule converge to a local Nash equilibrium. In *Advances in Neural Information Processing Systems 30*, 2017.

# A. Apéndices técnicos

## A.1. Algoritmo de entrenamiento

El Algoritmo 1 detalla el procedimiento de entrenamiento empleado para WGAN y WGAN-GP, incluyendo las actualizaciones del crítico y del generador, la aplicación del recorte de pesos o de la penalización de gradiente y el registro de las pérdidas. Se incluye para facilitar la reproducibilidad de la implementación descrita en la Sección 4.

**Algoritmo 1** Entrenamiento de WGAN y WGAN-GP

**Datos:** Método, $m = 64$, $n_{\text{critic}} = 5$, $K = 200\,000$, $c = 0{,}01$, $\lambda = 10$

Inicializar generador, crítico y optimizadores
Recuperar el estado si existe un checkpoint
**for** cada iteración pendiente hasta completar $K$ **do**
    **for** $t = 1, \ldots, n_{\text{critic}}$ **do**
        Tomar un lote real $\{x_i\}_{i=1}^{m}$
        Muestrear un lote de ruido $z_i \sim \mathcal{N}(0, I_{128})$
        Generar $\tilde{x}_i = G_\theta(z_i)$ sin propagar gradientes hacia $\theta$
        Calcular $L_D$ mediante la Ecuación (3)
        **if** el método es WGAN-GP **then**
            Muestrear $\epsilon_i \sim U[0, 1]$
            $\hat{x}_i \leftarrow x_i + \epsilon_i(\tilde{x}_i - x_i)$
            Añadir la penalización para obtener $L_D^{GP}$, Ecuación (6)
        Actualizar $w$ con la pérdida y el optimizador correspondientes
        **if** el método es WGAN **then**
            $w \leftarrow \text{clip}(w, -c, c)$
        Registrar la pérdida del crítico y, en WGAN-GP, la componente Wasserstein y la penalización
    Muestrear un nuevo lote de ruido
    Actualizar $\theta$ minimizando $L_G$, Ecuación (7)
    Registrar la pérdida del generador
    Guardar el estado cuando corresponda

## A.2. Evolución visual de las imágenes generadas

La Figura 4 muestra la evolución de las imágenes generadas por ambos modelos en los cuatro checkpoints evaluados: 1000, 10 000, 100 000 y 200 000 iteraciones. Para facilitar la comparación, ambos generadores reciben el mismo lote fijo de ruido y se muestran 64 imágenes por modelo.

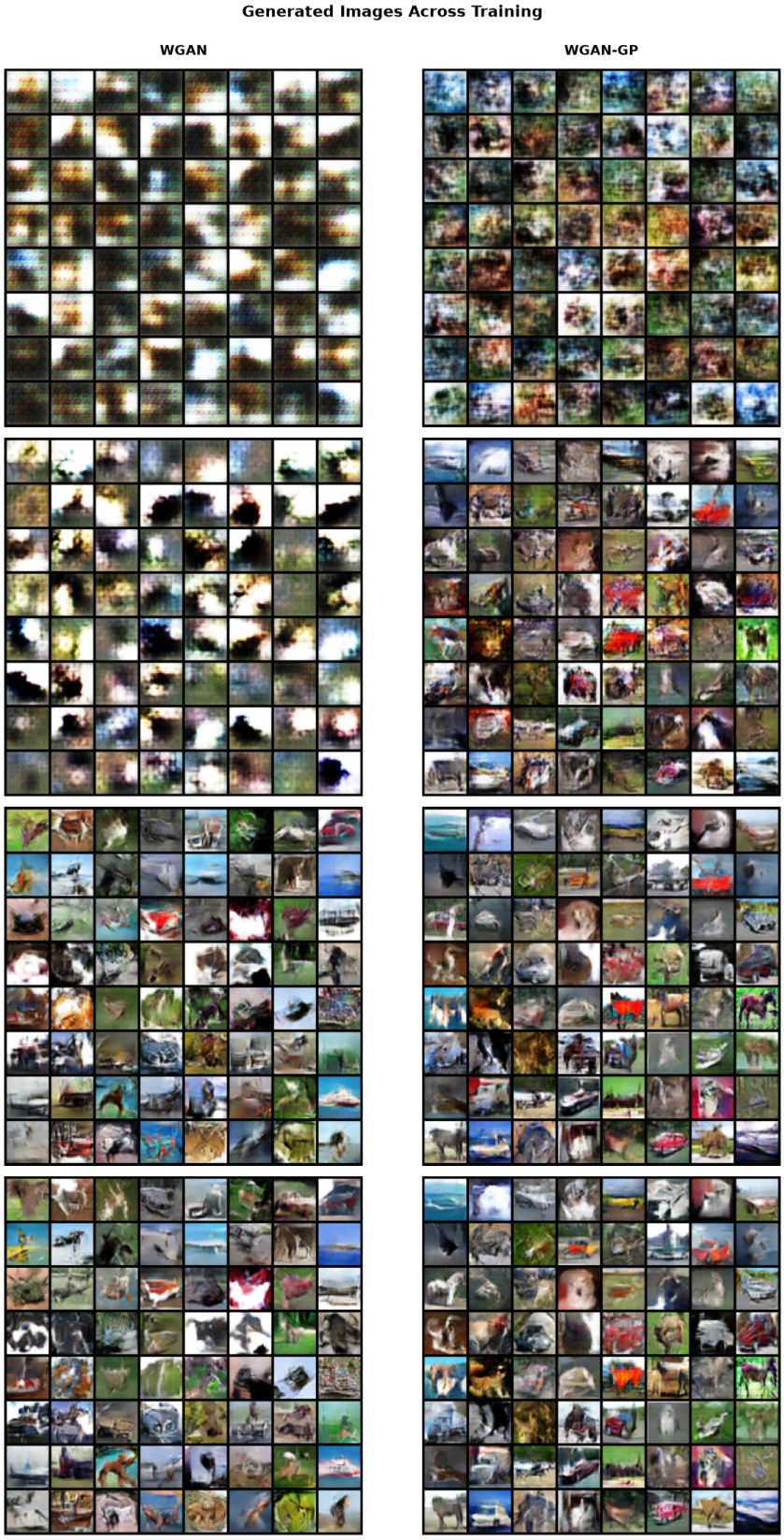



Figura 4: Gráfica de evolución de las imágenes generadas

## Contribuciones de los autores (CRediT)

- **Geer Wang Liu:** programó y entrenó WGAN-GP, revisó los artículos relacionados y redactó los apartados de trabajo relacionado y estado del arte y metodología.
- **Zhenhua Ye Xiang:** programó y entrenó WGAN, comparó los dos modelos y redactó los resultados, la discusión, las conclusiones y el trabajo futuro.
- **Antonio Peña Peña:** preparó la propuesta del proyecto, redactó la introducción y los objetivos, y revisó y corrigió el informe junto con los codigos.

## Declaración sobre el uso de herramientas de IA

Durante la preparación de este informe, los autores utilizaron GPT-6 Astra y Claude como apoyo para comprender el contenido teórico, resolver dudas, traducir textos entre español e inglés, mejorar la redacción, leer datos desde checkpoints y generar gráficas.

El uso de estas herramientas se limitó a explicar conceptos, aclarar cuestiones sobre los artículos consultados, proponer traducciones y mejoras en la claridad del texto, así como facilitar el procesamiento de datos y la visualización de resultados. Los autores revisaron y verificaron de forma independiente las respuestas y asumen plena responsabilidad sobre la exactitud, integridad, originalidad y contenido final del informe.